Roblox Up-and-Coming forecast — entry notes

What it is

`main.ts` (Node 22, standard library only, no network, deterministic) reads `history.ndjson` + `task.json` from `$MAKEFUN_INPUT_DIR` (default `/input`) and writes `forecast.json` to `$MAKEFUN_OUTPUT_DIR` (default `/output`).

Model — Monte Carlo rank diffusion

Each game's **log-rank** follows a random walk with drift, simulated hour by hour over the 72-hour scored window:

- **Drift** — the game's recent log-rank velocity over the last 48 h, recency-weighted, shrunk 0.75 toward 0, capped at 0.05/hour, decaying with a 24 h half-life as the window runs. This captures climbers (a game that went #25 → #12 in a day keeps some upward momentum) and fading former #1s.
- **Diffusion** — each hour, a step drawn from the *empirical* distribution of past per-hour log-rank steps (by rank band: top 10 / 11–30 / 31+), so the heavy tails of real rank jumps are kept. Scaled by 1.1. Plus a 2%/hour chance of an extra jump (sd 0.5 in log-rank) for sudden viral moves or updates.
- **Bounds** — log-rank reflects at 0 (rank 1) and is capped at log(130).
- **Age-out** — the chart only lists games published in the last 28 days, so when a game's `details.created` is available its probability is cut off at created + 28 days. (The practice history has no `details`, so this only activates on the final history, which the house recorder enriches.)
- **Candidates** — games seen in the last 7 days or on the last chart (~150–200 games). 1,500 seeded simulations × 72 hours; the fraction of simulated hours a game holds each place is its forecast probability.
- **Unnamed share** — each place's named probabilities are scaled to sum to 0.85, leaving 0.15 for games that appear out of nowhere. Brand-new games do reach #1 on this chart (e.g. "Do Not Stop." had no history at all and took #1 in the practice window), so this insurance matters.
- **top10** — greedy by highest probability per place, distinct games.

Determinism: one fixed seed (mulberry32); no wall-clock, no unseeded randomness. Two runs on the same history are byte-identical (verified).

Why not plain persistence

The house baseline bets the chart will not move (0.5 on the current #k at place k). The chart moves fast: between consecutive reads ~7 h apart, only ~87% of the top 10 stays in the top 10, and the #1 changed 13 times in the archive's 45 reads. Over a 72 h window the history-end #1 ("Survive Your Ideas") held #1 in **none** of the 10 practice-window reads. Persistence is confident and wrong; spreading calibrated probability across recent #1s, climbers and neighbours scores far better.

Local test results

Scored with the official log-score rule (mean over reads of the sum over places 1–10 of ln max(P, 0.0001); a place's unnamed share is spread over the games on that read's chart not named there). The practice answer was rebuilt from the public archive (github.com/Kenny-Dhub/roblox-snapshots, the 10 reads 2026-11-08T00:37Z – 2026-11-10T17:51Z); the scorer reproduces the published baseline exactly (-43.512039, 6.4 hits, 53.5 agreement).

Test	Baseline	This entry
Practice window (10 reads, train = 34-read history)	-43.512	**-34.66**
Split 1: train first 24 reads, test last 10 reads	-56.911	**-43.01**
Split 2: train first 28 reads, test last 6 reads	—	**-30.72**

Practice-window detail for this entry: typical probability per place 0.031 (baseline 0.013), 7.2 top-10 hits (baseline 6.4), 62.0 agreement (baseline 53.5).

Tuning: 17 variants were compared on the practice window and both splits (drift shrink 0 / 0.5 / 0.75 / 1.0; diffusion scale 0.07–0.13; unnamed share 0.06–0.20; drift half-life 12 / 24 / 48 h; with/without jumps and mean reversion; three seeds). Pure persistence and no-drift diffusion were clearly worse; every drift 0.75–1.0 variant landed within ~0.6 nats of each other across the three tests combined (MC seed noise alone is ~±0.4), so the shipped parameters (drift 0.75, diffusion 0.11, unnamed 0.15, half-life 24 h) are a robust middle choice, not a knife-edge optimum. Final parameters are in the header of `main.ts`.

Validation run

```
mkdir -p out && MAKEFUN_INPUT_DIR=<kit>/input MAKEFUN_OUTPUT_DIR=out node main.ts
```

- Runs in ~5 s (limit 300 s), 1 CPU, stdlib only, no network.
- `forecast.json`: 28,780 bytes (limit 1,048,576), exactly two keys (`probabilities`, `top10`); 561 rows; every place sums to 0.85 (≤ 1), at most 77 rows per place (limit 100), all p in [0, 1], no duplicate (game, place) pairs, 10 distinct universe_ids in `top10`.

- Two consecutive runs produce byte-identical output.

Files

- `main.ts` — the entry program (run as `node main.ts` from the repo root).
- `package.json` — `{"type": "module"}` so Node parses `main.ts` as ESM without a warning; no dependencies.
- `LICENSE` — MIT (copied from the kit's starter).
- `NOTES.md` — this file.